

基于视频大模型的通用视频高光剪辑

复赛技术方案

参赛团队编号: AIC-2026-12102510

团队名称: 根目录

队长姓名: 王逸轩

提交日期: 2026 年 10 月 9 日

一、摘要

本方案面向“给定一段视频与目标画幅比例，输出高光片段内每一帧的构图裁剪框”这一任务，构建了一条不依赖任何训练、可完全复现的流水线。核心结论有三条，均由本地自造验证集与线上真实反馈双重印证：

- 时间维不做取舍。**评测指标为逐视频的 $F1=2S/(N_{pred}+N_{gt})$ ，多预测只会线性摊薄分母，漏掉真值帧却会直接丢匹配。因此对每一帧都输出预测框 ($\tau=1$)，在本地与线上均显著优于任何抽样窗口。
- 空间维用一个“逐视频恒定偏置”打败居中框。**对每帧所有 person 检测按 $conf \times area$ 加权池化得到该帧的重心，再取逐视频中位数，作为该视频唯一的构图中心常数。此单条改动在本地 val 上贡献 +0.0526，线上贡献 +1.80。
- 用“杂乱度”门控兜住下界。**当画面内人数过多、加权中心失去意义时，回退为居中裁剪。门控采用绝对杂乱刻度而非分位数，因为线上测试集的拥挤度只有本地验证集的一半。被门控的视频输出与上一版冠军逐像素相同，故该改动是一条“免费期权”：单条下界即现冠军，上行 +0.06。

线上排行榜实测：居中基线 31.05 → 红框 + 门控 **32.85**（满分 100 制）。全流程唯一加载的模型权重为 YOLO11n (5.26 MB)，远低于 9 GB 上限。

二、赛题与评测指标

2.1 输出形式

评测脚本对每条视频计算其预测集与真值集的匹配情况。预测以 JSONL 逐行给出，每行包含 video_id、targetRatioWH、model_size_mb 及 predictions 列表；predictions[i] 为 {"frame": 全局源帧号,

"bboxes": [x, y, w]} ——裁剪框用三元组表示，高度由 $h = w \cdot \text{target_h} / \text{target_w}$ 唯一确定，故不提交高度。

2.2 官方指标

逐视频计算 $F_{\text{video}} = 2S / (N_{\text{pred}} + N_{\text{gt}})$ ，其中 S 为该视频预测帧与真值帧在框重叠达标（IoU 阈值）前提下的匹配数， N_{pred} 、 N_{gt} 分别为预测帧数与真值帧数；最终得分取全部视频的均值。该式有一个直接推论：增加预测帧只线性增大分母，而漏掉真值帧会同时减少分子与改变匹配结构，代价更高。这一点决定了本方案时间维的全部设计。

2.3 初赛 / 复赛的排序口径差异

初赛排行榜依据原始 F_{video} 排序，用于算法验证；复赛依据模型大小加权后的 F_{final} 排序，用于省赛排名与决赛晋级。因此复赛除精度外还需控制 `model_size_mb`（本次推理实际加载的全部权重、解压后总大小），并要求 JSONL 各行取值一致。本方案权重总量极小，此维度不构成负担。

三、数据

3.1 复赛测试集

赛方提供的复赛测试集共 426 条视频，编号 0–425 连续，每条配一份 `jsonl/<id>.jsonl` 声明目标画幅（16:9 共 303 条、9:16 共 123 条）。经逐文件校验，初赛的 174 条（编号 0–173）与复赛对应编号的视频字节完全相同（md5 一致、分辨率/帧数/画幅声明一致），即复赛测试集是初赛测试集的超集。本方案据此对 0–173 复用既有结果、对 174–425 新跑，二者合并即为 426 条完整提交。

3.2 本地验证集（代理指标）

测试集无真值，故本方案自建了一个 57 条的本地验证集：从公开的高光数据集按标注切出片段，构造与赛题一致的“目标画幅 + 逐帧裁剪框”监督。评测器与官方指标同构。需要明确的是，本地 val 只是代理指标：其来源、剪辑结构、拥挤度分布与复赛测试集均不同，数值不可直接迁移，仅用于相对排序与超参选择（迁移折算见 § 八）。

四、方法总览

整条流水线分三步，前两步决定“留哪些帧、框多大”，第三步只决定“框沿自由轴放在哪里”。

阶段	输入	输出 / 规则
① 时间定位 + 框尺寸	原视频、目标画幅	逐帧保留 ($\tau=1$)，丢弃末 8% 帧；框尺寸由目标画幅唯一决定
② 红框臂	逐帧 person 检测 (YOLO11n)	逐视频恒定构图中心： <code>conf×area</code> 加权池化的逐帧重心 → 取视频内中位数
③ 杂乱门控	逐帧 person 检出数	平均检出数 > 2.0 的视频回退居中框；否则沿自由轴平移至 ② 的中心

全流程不含任何模型训练，唯一的神经网络是 YOLO11n 目标检测器，且它只被用于统计“画面里有多少人”，不参与任何坐标回归。这意味着方案没有过拟合到某个训练集的风险，且模型体积、推理成本都可忽

略。

五、关键设计决策与实验依据

5.1 时间维：为什么“每帧都输出”

τ （保留帧比例）扫描（居中框、离线模拟、指标单调）如下：

$\tau=0.20 \rightarrow 0.1551$	$\tau=0.40 \rightarrow 0.2486$	$\tau=0.60 \rightarrow 0.3157$	
$\tau=0.35 \rightarrow 0.2301$	$\tau=0.50 \rightarrow 0.2827$	$\tau=0.70 \rightarrow 0.3434$	$\tau=1.00 \rightarrow 0.3575$

$\tau=1$ 时召回率为 1.0000（不漏任何真值帧），代价是精确率降至 0.386——但由于分母是 $N_{\text{pred}}+N_{\text{gt}}$ ，多预测被线性摊薄，而漏检直接损失匹配，故全帧策略在整条曲线上单调最优。这与“先判断哪些帧是高光再裁剪”的直觉相反，是本题指标结构决定的。

5.2 尾部丢弃：8% 是推出来的，不是调出来的

统计真值帧在片中的位置分布发现，前 88% 的“该位置帧属于高光的视频比例” p 稳定在 0.40–0.60，而末 12% 单调掉到 0.02（真值终点中位落在第 87% 帧，片尾留有约 13% 空白）。由指标结构，保留帧 f 的边际条件为

保留帧 $f \iff p_f > F1 / (2a) \approx 0.28$

p 恰好在第 92 百分位处穿过该阈值，故应丢弃末 8%。扫描验证一致（丢 8% $\rightarrow$ 0.3683，为峰值）。注意这是单向的：丢弃片头是单调亏损，说明该结构真实存在而非噪声。**诚实声明：**此先验的代价与收益大致对称（若测试集真值在时间上均匀，丢末 8% 损失约 -0.0085、收获约 +0.0095），是一次机制驱动的赌注，而非拟合。

5.3 空间维基线：为什么居中框很强

对多数视频，真值裁剪框已接近画面中心——本地验证集上真值中心的逐帧中位偏移仅约 -2px。因此“始终居中”是一个被低估的强基线。内容信号（帧差、清晰度、曝光）的逐帧判别 AUC 全部在 0.48–0.59 之间（0.5 无信息），其中曝光甚至反相关；逐帧跟踪相对居中框的增益仅 +0.0016，且 14 条抽样子集里有 8 条输出与居中框逐像素相同（检测器无输出即回退中心）。结论：逐帧的内容信号在本题近似抛硬币，真正的增量必须来自“每条视频一个的、与内容相关的空间偏置”。

5.4 红框臂：逐视频恒定偏置

做法：对每一抽样帧，将该帧所有 $cls=="person"$ 的检测按权重 $w = conf \times area$ 沿自由轴（16:9 源为横向 x ，9:16 源为纵向 y ）加权平均，得到该帧的池化重心；再对整条视频取中位数，作为该视频唯一的构图中心常数，据此把裁剪框沿自由轴平移（平移量 $o = \text{round}(\text{clip}(c \cdot W - cw/2, 0, W - cw))$ ）。

臂	本地 val F1	Δ vs 居中冠军
居中框（上一版冠军）	0.3688	—
红框臂（每视频常数中心，无门控）	0.4214	+0.0526

选择“逐视频常数”而非“逐帧跟随”，是因为逐帧中心本身噪声极大（视频内抖动 $sd \approx 0.117$ ），而其中真正可用的成分是**视频间的恒定偏置**（ $sd \approx 0.146$ ，与真值偏置相关 0.49）。取中位数 即是在丢弃高频抖动、只保留稳定偏置。

5.5 为什么不用跟踪器

同一权重、同一批视频、同一抽样步长下，引入 ByteTrack 反而**减少 19% 的 person 检出**（8546 → 6922）。原因是未关联上的框会被丢弃，而红框臂的全部信息恰恰来自“把一帧内所有 person 框加权平均”——丢框即丢信息。实测红框臂由 0.4214 降至 0.4131。故**不使用任何跟踪器**，顺带的好处是线上只需跑一遍纯检测前向，无需维护跟踪状态。

5.6 杂乱门控与“绝对阈值”

红框臂的收益并非全体均匀：它在“人多杂乱、没有唯一主体”的视频上会给出无意义的中心。按每抽样帧平均 person 检出数分档统计红框臂的 Δ ：

每抽样帧平均人数	val 条数	红框臂 Δ
0.0-0.5	6	+0.0085
0.5-1.0	9	-0.0114
1.0-1.5	18	+0.1874
1.5-2.0	6	-0.0277
2.0-3.0	6	+0.0082
3.0-6.0	6	-0.0299
6.0+	6	-0.0835
全部	57	+0.0443

可见收益集中在 1.0-1.5 这一档（+0.187，由 18 条扛起整体），损害集中在远端尾部（ ≥ 3.0 两档为负）。因此把“平均检出数 > 2.0 ”判定为拥挤，这些视频**回退为居中裁剪**；其余走红框。门控手臂的关键性质是**下界锚定**：被门控视频的输出与上一版冠军逐像素相同，故最坏情况下整份提交等于 已在服务器上的版本，只存在上行空间。

为什么用绝对刻度而非分位数。本地验证集内部做留出检验时，两半分布相同，“取前 q 分位”与“取绝对阈值”必然等价，该实验从未检验过分布迁移。一旦对比测试集，两者立刻分开：本地 val 拥挤度均值 2.11 / 中位 1.18；初赛测试集（174 条）均值 0.95 / 中位 0.86，即**只有本地一半拥挤**——若沿用本地 $q=0.30$ 的分位切点（2.014），在初赛测试集上只会圈住 约 10% 的视频，而按 rank 定义照样切掉 30%，恰好切进 1.0-1.5 这条赚钱带。复赛全量 426 条的分布则更接近本地（均值 1.80 / 中位 0.83），但带有很重的长尾（ $p_{90} \approx 4.9$ 、 $p_{99} \approx 14.4$ 、最大 ≈ 17.9 ），且仍有 40.8% 的视频落在几乎无人的 < 0.5 档。由于**“损害 ~ 绝对杂乱度”**是一个固定形状的函数、与数据集无关，门控采用**绝对阈值 2.0 而非分位数**，才能在分布变化时既**保住赚钱带、又挡住长尾**。在本次复赛提交中，2.0 的绝对阈值使 19.5% 的视频 因拥挤、另有一批因完全无 person 检出而回退居中框（合计 145/426）。

结果表（本地 val 端到端，帧集与框尺寸一字不动，只改自由轴偏移）：
居中框 0.3688 → 红框无门控 0.4214 → 红框 + 门控 0.4302（阈值 2.0，约 17% 视频退回）。

六、本地验证集与评测器

由于测试集无真值，全部超参选择都建立在一套“自造真值 + 与官方同构的评测器”之上：

- **构造一致性**：本地 val 的目录结构、画幅声明、裁剪框表示（三元组、不提交高度）与赛题完全一致，可直接用同一套生成脚本处理。
- **评测器自检**：提交前必跑 `eval_local.py validate`，检查行数等于索引条数、无越界帧号/坐标、无 NaN、`bboxes` 恰为 3 个整数、`model_size_mb` 存在且各行一致；输出 `RESULT: PASS` 方可提交。
- **双盲消融**：任何改动都通过“只变一个变量”的覆盖式改写实现——帧集与框尺寸保持字节级不变，仅改变自由轴偏移，从而保证每次 Δ 都可归因。

七、复赛提交格式与 model_size_mb 口径

提交的“赛题结果文档”为 426 行 JSONL，每行结构：

```
{ "video_id": "0", "targetRatioWH": [16, 9], "model_size_mb": 5.26,
  "predictions": [ { "frame": 0, "bboxes": [0, 102, 720] }, ... ] }
```

`model_size_mb` 口径为“本次推理实际加载的全部模型权重的解压后总大小（MB）”，全部 426 行取同一值：

权重文件	磁盘大小	解压后大小	填报值
yolo11n.pt（唯一权重）	5,613,764 B	5,520,527 B	5.26 MB

`yolo11n.pt` 本身是 zip 容器（STORE 无压缩），磁盘体积含目录开销，故“解压后”为各 entry 的 `file_size` 之和 = 5,520,527 B = 5.26 MiB。方案不含 Adapter/LoRA、不含分割或跟踪模型，权重总量远低于 9 GB 上限。

八、结果

8.1 本地验证集（57 条，代理指标）

配置	τ	框	F1
先验窗基线	0.40	居中	0.2577
全帧基线（上一版冠军）	1.00	居中	0.3688
红框臂	1.00	每视频常数	0.4214
红框 + 杂乱门控（本方案）	1.00	门控	0.4302

8.2 线上排行榜（真实反馈，非代理）

提交	本地 val F1	线上得分
先验窗基线 ($\tau=0.40$)	0.2577	24.85
全帧居中基线 ($\tau=1.0$)	0.3688	31.05
红框 + 杂乱门控	0.4302	32.85

最重要的一条工程教训（迁移折算）：本地 val 的 Δ 要打约三折。例如框臂在本地 val 上贡献 +6.14（百分制），线上实测只有 +1.80（比值约 0.29）。这与其来源一致：复赛测试集只有本地一半拥挤，且约 36% 的视频落在本地几乎无样本的档位。因此本报告对任何本地改进均按大约 $\times 0.3$ 估计线上预期，不以本地 Δ 直接宣称线上收益。

8.3 本次复赛提交概况（无真值，仅规模与门控统计）

项	数值
视频条数	426 (16:9 共 303, 9:16 共 123)
预测条目总数	215,428
走红框的视频	281
回退居中框的视频（门控）	145 (拥挤超阈 83 + 完全无 person 检出 62)
model_size_mb	5.26 (426/426 行一致)
自检结果	VALIDATE PASS, violations = 0

九、局限与诚实声明

- 本地分是代理指标。** val 与测试集来源不同，绝对数值不可迁移，仅用于相对排序。
- 无证据档位。** 复赛测试集约 41% 的视频落在“每抽样帧平均人数 < 0.5 ”档，而本地该档样本极少，红框臂在这一档的作用缺本地支撑；若该机制完全不迁移，红框的期望增益退化为 0——但门控把下界锚定在上一版冠军。
- 换轴未本地验证。** 复赛 123 条为 9:16 源（自由轴为 y），本地 val 绝大多数为横向（自由轴 x）。“杂乱度”的定义与轴无关，但这一支仅能等线上反馈。
- 首尾结构先验。** 丢末 8% 依赖“片尾有空白”的剪辑结构，若测试集真值在时间上均匀，该改动大致打平（见 § 5.2）。
- 0/0 约定未知。** 若某视频真值为空且我们输出为空，F1 未定义；评测器同时给出三种约定下的结果。

十、已实测为负的路线（供评委参考，避免重复投入）

按赛题“思路引导”中给出的方向，我们逐一做了实测（均以本地 val 为判据），结果如下：

- 多模态大模型直接定位（VLM）：**用开源 VLM 做高光区间与构图框预测，其空间阶段为净损失（把大量本已居中的框推离中心），时间区间召回率仅 0.489，低于同尺寸随机窗的 0.494。**结论：不如不加模型。**

- **分割模型生成候选 mask (SAM)**: 用 SAM 的 mask 质心替代 person 框质心。逐视频偏置 反而更差 (0.080 vs person 框 0.068), 纯 SAM 臂端到端仅 0.368 ($\approx$ 居中基线); 与 person 框混合虽在无门控时有 +0.0025, 但在门控臂上增益精确归零; 以 SAM 统计量作门控特征亦劣于 person 计数。 **结论**: 分割器读得出像素, 但读不出构图意图。

两者的共同结论指向同一点: 本任务的真值是“**构图意图**”(framing) 而非“画面主体”(subject) —— 例如讲台场景的框是套住讲者、骑手场景的框整体留出视线方向、新闻场景的框落在字幕栏。这类约定必须由 **经过训练、能直接输出构图中心的模型** 给出, 无监督/通用检测器与分割器都给不出。本方案的取舍是: 在具备该能力之前, 先把“不用模型也能确定”的部分(时间结构、居中基线、逐视频空间偏置、杂乱门控)做到最优, 保证一个稳固的下界。

附录 A: 提交物清单

AIC-2026-12102510-根目录-复赛-王逸轩/	
├─ AIC-2026-12102510-技术方案.PDF	← 本文件
├─ AIC-2026-12102510-模型文件与运行环境/	
│ └─ yolol1n.pt	推理唯一加载的权重 (5.26 MB 解压后)
│ └─ requirements.txt	依赖与版本
│ └─ 运行环境说明.md	环境、模型清单与 model_size_mb 口径
├─ AIC-2026-12102510-算法代码/	
│ └─ baseline_cv.py	居中基线生成
│ └─ yolo_experiment.py	检测缓存构建
│ └─ emit_gated.py	红框臂 + 门控叠加、写 model_size_mb
│ └─ stage_data.py	几何与元信息公共工具
│ └─ eval_local.py	本地评测 / 提交自检
│ └─ fu_index.json / fu_metadata.json	426 条索引与元信息
│ └─ 运行说明.md	一键复现步骤
└─ AIC-2026-12102510-赛题结果文档/	
└─ predictions.jsonl	426 行, 每行含 model_size_mb

附录 B: 复现命令

```
# 依赖
pip install -r AIC-2026-12102510-模型文件与运行环境/requirements.txt

# 1) 居中基线 ( $\tau=1$ , 逐帧保留, drop-tail 0.08)
python3 baseline_cv.py --index fu_index.json --video-dir <TEST>/video \
    --keep-ratio 1.0 --alpha 0 --no-yolo --out fu_base.jsonl

# 2) 检测缓存 (一次 GPU 前向, stride=5)
python3 yolo_experiment.py --build-cache --index fu_index.json \
    --video-dir <TEST>/video --metadata fu_metadata.json \
    --cache fu_yolo_test_cache.json --stride 5 --device 0

# 3) 红框 + 杂乱门控叠加, 写入 model_size_mb
python3 emit_gated.py --base fu_base.jsonl --cache fu_yolo_test_cache.json \
    --metadata fu_metadata.json --threshold 2.0 \
    --weights AIC-2026-12102510-模型文件与运行环境/yolol1n.pt --out predictions.jsonl

# 4) 提交前自检
python3 eval_local.py validate --index fu_index.json --pred predictions.jsonl
```